CSE22AIE205 · PYTHON PROGRAMMING

Object Orientation, Exceptions, Regular Expressions & Threads

- Session 1: Object-Oriented Programming in Python
- Session 2: Exception Handling
- Session 3: Regular Expressions + Multithreaded Programming

Think Python: How to Think Like a Computer Scientist, 2nd Ed. — Allen B. Downey

Module Roadmap

1

Section 1 — Python Object-Oriented Programming

Classes, objects, constructors, inheritance, polymorphism, encapsulation

2

Section 2 — Exceptions

try / except / else / finally, raising and creating custom exceptions

3

Section 3 — Regular Expressions

Pattern matching, groups, substitution, validation with the re module

4

Section 4 — Multithreaded Programming

Threads vs processes, the GIL, race conditions, locks, thread pools

SECTION 01

Python Object-Oriented Programming

Classes, objects, and building reusable, real-world models in code



Procedural vs. Object-Oriented Thinking

Procedural Style

- ✓ Data and functions are separate
- ✓ Functions operate on data passed to them
- ✓ Harder to model real-world entities
- ✓ Reuse means copying similar functions
- ✓ Example: separate variables for a student's name, marks, and a function calc_grade(name, marks)

Object-Oriented Style

- ✓ Data (attributes) and behaviour (methods) live together
- ✓ An object bundles state + operations on that state
- ✓ Naturally models real-world things: Student, Car, BankAccount
- ✓ Reuse via inheritance — extend, don't copy
- ✓ Example: a Student object that knows its own name, marks, and calc_grade()

CORE IDEA

Classes and Objects

- A class is a blueprint — it defines what attributes (data) and methods (behaviour) its objects will have
- An object (or instance) is one concrete thing built from that blueprint

Class Student defines the blueprint; s1 = Student("Ravi") creates one real object

- Every object created from a class has its own copy of instance attributes, but shares the same methods
- In Think Python's terms: a class is a programmer-defined type, and an object is a value of that type

Analogy

Blueprint = Class

The architectural drawing of a house — defines rooms, doors, layout

House = Object

Each house built from the drawing is a separate object with its own address

Many objects, one class

You can build 100 houses (objects) from a single blueprint (class)

LIVE DEMO

Defining a Class and Creating an Object

```
class Point:
    """Represents a point in 2-D space."""
    pass

p = Point()
p.x = 3.0
p.y = 4.0

print(p)
print(f"({p.x}, {p.y})")
```

OUTPUT

```
<__main__.Point object at
0x7f2a1c0b3d90>
(3.0, 4.0)
```

- class Point: creates a new type — Point is now a blueprint
- p = Point() instantiates an object; attributes x and y are attached directly onto it
- The default print(p) shows the object's type and memory address — not very readable yet



CONSTRUCTOR

The _init_ Method

- __init__ is a special method Python calls automatically when an object is created
- It initializes instance attributes so every object starts in a valid, predictable state
- The first parameter is always self — a reference to the object being created
- Default parameter values make arguments optional, e.g. def __init__(self, x=0, y=0)
- Without __init__, you would have to set every attribute manually after creating each object

self, explained

Not a keyword
self is just a convention — Python passes the object automatically

Object reference
self.x = x means "store x on THIS particular object"

Always first
Every instance method's first parameter is self

LIVE DEMO

Using __init__ with Parameters

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

p1 = Point(3, 4)
p2 = Point()

print(p1.x, p1.y)
print(p2.x, p2.y)
```

OUTPUT

```
3 4
0 0
```

- p1 = Point(3, 4) calls __init__(p1, 3, 4) behind the scenes
- p2 = Point() uses the default values x=0, y=0

Instance Attributes vs. Class Attributes

Instance Attribute

- ✓ Defined with self.name = value inside a method
- ✓ Belongs to one specific object
- ✓ Every object has its own independent copy
- ✓ Used for data that varies per object (name, marks, balance)

Class Attribute

- ✓ Defined directly inside the class body
- ✓ Shared by ALL objects of that class
- ✓ Changing it via ClassName.attr affects every instance
- ✓ Used for data common to all objects (a counter, a constant, a default)

LIVE DEMO

Class Attribute vs. Instance Attribute

```
class Robot:
    population = 0        # class attribute

    def __init__(self, name):
        self.name = name    # instance attribute
        Robot.population += 1

r1 = Robot("R2D2")
r2 = Robot("C3PO")

print(r1.name, r2.name)
print("Total robots:", Robot.population)
```

OUTPUT

```
R2D2 C3PO
Total robots: 2
```

- population belongs to the Robot class itself, not to r1 or r2 individually
- Every time __init__ runs, it increments the ONE shared counter

BEHAVIOUR

Methods — Giving Objects Behaviour

- A method is a function defined inside a class; it operates on a specific object's data
- Called using dot notation: object.method(args) — Python passes the object as self automatically
- Methods can read and modify the object's own attributes via self
- Methods can also return computed values based on the object's state
- This groups related behaviour with the data it acts on — a core OOP principle

Method call sugar

t.print_time()
Python rewrites this as Time.print_time(t) internally

self = t
Inside the method, self refers to the object t

LIVE DEMO

Methods in Action — a Time Class

```
class Time:
    def __init__(self, hour=0, minute=0, second=0):
        self.hour = hour
        self.minute = minute
        self.second = second

    def print_time(self):

print(f"{self.hour:02d}:{self.minute:02d}:{self.second:02d}
")

    def time_to_int(self):
        return self.hour*3600 + self.minute*60 +
self.second

t = Time(9, 5, 30)
t.print_time()
print("Seconds since midnight:", t.time_to_int())
```

OUTPUT

```
09:05:30
Seconds since midnight: 32730
```

- print_time() formats and displays the object's state
- time_to_int() derives a new value FROM the object's attributes



DUNDER METHODS

__str__ and Readable Objects

- By default, printing an object shows an unhelpful <...object at 0x...> address
- __str__(self) lets you define exactly what print(obj) or str(obj) should show
- Python calls __str__ automatically — you never call it directly
- This is one of many "dunder" (double-underscore) special methods Python looks for
- __repr__ is a close cousin, meant for an unambiguous, developer-facing representation

LIVE DEMO

Implementing __str__

```
class Time:
    def __init__(self, hour, minute, second):
        self.hour, self.minute, self.second = hour,
minute, second

    def __str__(self):
        return
f"{self.hour:02d}:{self.minute:02d}:{self.second:02
d}"

t = Time(9, 5, 30)
print(t)                    # uses __str__
automatically
print(f"Class time is {t}")
```

OUTPUT

```
09:05:30
Class time is 09:05:30
```

- print(t) now calls t.__str__() behind the scenes and shows a clean, readable value



DUNDER METHODS

Operator Overloading

- Operator overloading lets custom objects respond to +, -, ==, <, and other operators
- __add__(self, other) defines what p1 + p2 should do for your class
- Other common ones: __eq__ (==), __lt__ (<), __len__ (len()), __getitem__ (indexing)
- This is why Python's own types feel consistent — lists, strings, and numbers all overload + differently

LIVE DEMO

Overloading + with __add__

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(3, 4)
p2 = Point(1, 2)
p3 = p1 + p2
print(p3)
```

OUTPUT

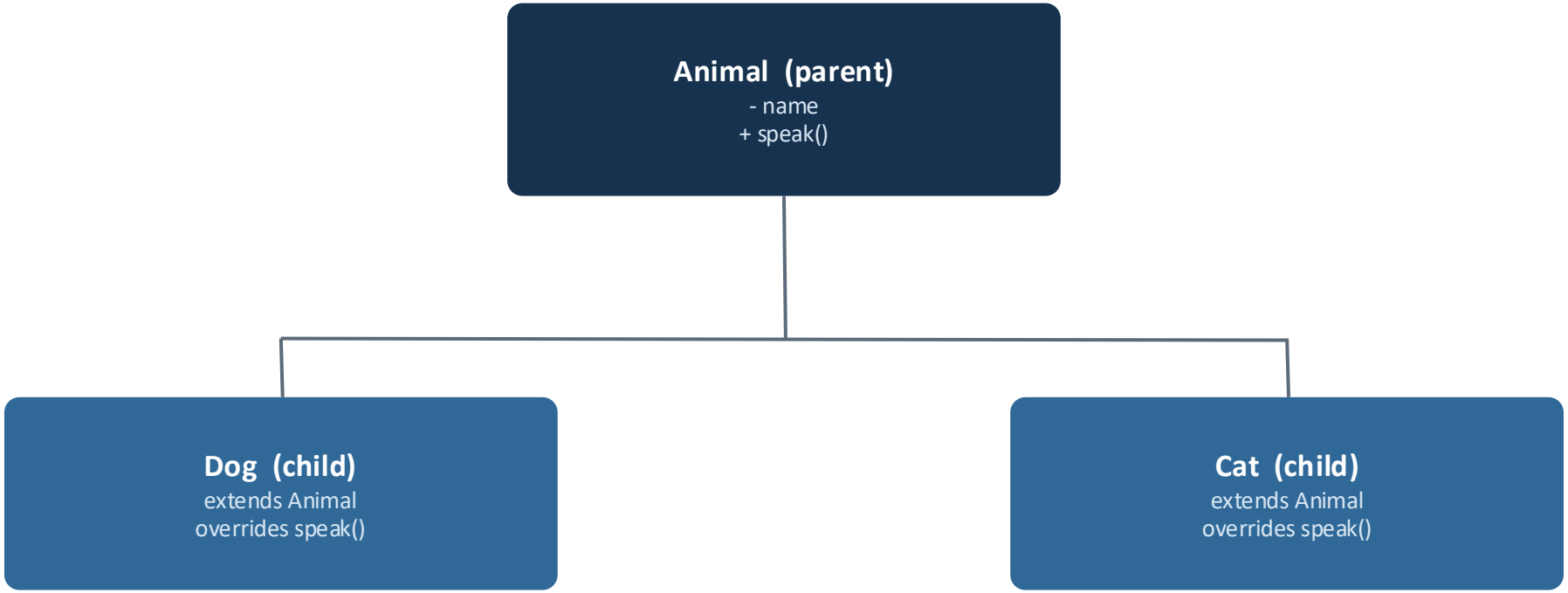
```
(4, 6)
```

- p1 + p2 is translated by Python into p1.__add__(p2)
- The result is a brand-new Point object — the originals are unchanged

REUSE & EXTEND

Inheritance — Building on Existing Classes

A child class automatically gets all attributes and methods of its parent, and can add or override its own.




```
class Dog(Animal):
```

```
class Cat(Animal):
```

LIVE DEMO

Inheritance in Practice

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

d = Dog("Rex")
c = Cat("Whiskers")
print(d.speak())
print(c.speak())
```

OUTPUT

```
Rex says Woof!
Whiskers says Meow!
```

- Dog(Animal) and Cat(Animal) inherit __init__ from Animal — no need to rewrite it
- Each overrides speak() with its own version



MANY FORMS

Polymorphism

- Polymorphism: the same method call behaves differently depending on the object’s actual class
- Code written for the parent type (Animal) works automatically with any child (Dog, Cat, ...)
- Enables writing general-purpose code: a single loop can call .speak() on any list of Animals
- This is what makes large systems extensible — new subclasses "just work" with existing code

LIVE DEMO

Polymorphism — One Loop, Many Behaviours

```
animals = [Dog("Rex"), Cat("Whiskers"), Animal("Generic")]

for a in animals:
    print(a.speak())
```

OUTPUT

```
Rex says Woof!
Whiskers says Meow!
...
```

- The loop doesn’t know or care which subclass each object is — a.speak() always calls the RIGHT version



DATA PROTECTION

Encapsulation

- Encapsulation means bundling data with the methods that operate on it, and restricting direct access
- Convention: _name (single underscore) = "protected", intended for internal use only
- Convention: __name (double underscore) triggers name mangling — Python renames it to _ClassName__name
- This discourages (but doesn’t forcibly prevent) accidental access from outside the class
- Access is instead offered through controlled methods — often called getters and setters

LIVE DEMO

Encapsulation with a Private Attribute

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())
print(acc._BankAccount__balance)    # name-mangled access
```

OUTPUT

```
1500
1500
```

- __balance is only meant to change through deposit() — controlled, validated access
- The mangled name still works if accessed directly, but doing so breaks encapsulation on purpose

METHOD TYPES

Instance, Class, and Static Methods

@classmethod

- ✓ First parameter is cls, the class itself (not an object)
- ✓ Can access/modify class-level state
- ✓ Often used as alternative constructors
- ✓ e.g. Circle.unit_circle() builds a Circle without calling Circle(1) directly

@staticmethod

- ✓ Takes no automatic self or cls parameter
- ✓ Behaves like a plain function placed inside the class for organisation
- ✓ Cannot access instance or class state
- ✓ Used for utility/helper logic related to the class

LIVE DEMO

classmethod and staticmethod

```
class Circle:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    @staticmethod
    def is_valid_radius(r):
        return r > 0

    @classmethod
    def unit_circle(cls):
        return cls(1)

c = Circle.unit_circle()
print(c.radius)
print(Circle.is_valid_radius(5))
print(Circle.is_valid_radius(-2))
```

OUTPUT

```
1
True
False
```

- Circle.unit_circle() is called on the CLASS, yet returns a new object
- Circle.is_valid_radius(5) needs no object at all to be called

RECAP

Section 1 Recap — OOP in Python

- ✓ Classes are blueprints; objects are instances built from them
- ✓ __init__ initializes instance attributes when an object is created
- ✓ Instance attributes vary per object; class attributes are shared by all
- ✓ Methods bundle behaviour with data; self refers to the calling object
- ✓ __str__ and other dunder methods customize how built-in operations behave
- ✓ Inheritance, polymorphism, and encapsulation let code be reused, extended, and protected

SECTION 02

Exception Handling

Writing Python programs that fail gracefully instead of crashing



Syntax Errors vs. Exceptions

Syntax Error

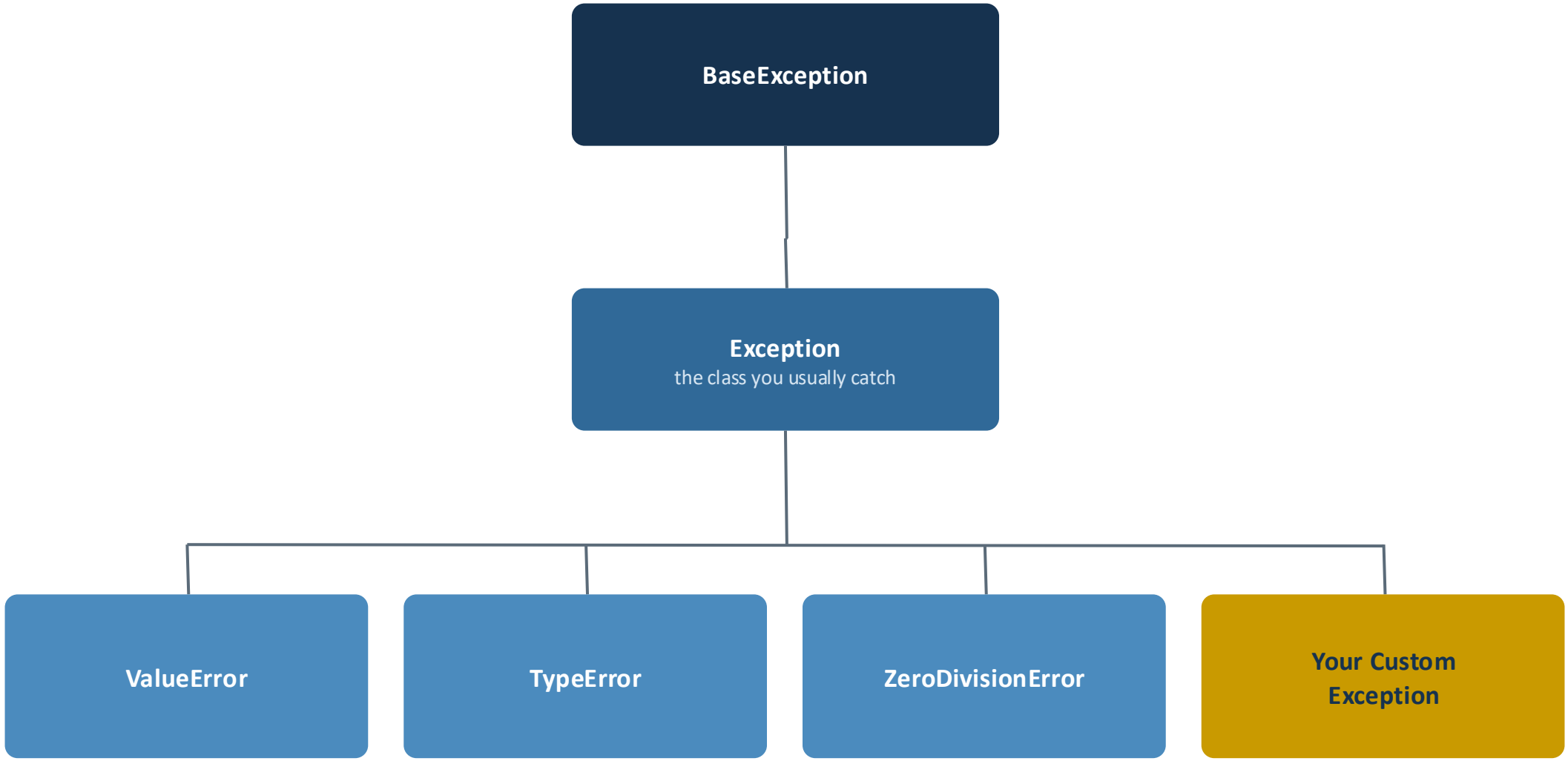
- ✓ Detected before the program even runs
- ✓ Caused by invalid Python grammar
- ✓ e.g. missing colon, unmatched bracket
- ✓ Must be fixed in the code itself — cannot be "handled"

Exception (Runtime Error)

- ✓ Occurs WHILE the program is running
- ✓ Caused by a valid statement hitting an invalid situation
- ✓ e.g. dividing by zero, opening a missing file, bad user input
- ✓ CAN be anticipated and handled using try / except

CLASS HIERARCHY

Python's Built-in Exception Hierarchy



LIVE DEMO

A Basic try / except Block

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")

print("Program continues...")
```

OUTPUT

```
Error: Cannot divide by zero!
Program continues...
```

- Without try/except, this would crash the whole program with a traceback
- Catching the specific exception lets execution continue normally afterward



HANDLING

Catching Multiple Exception Types

- A try block can be followed by several except clauses, each for a different exception type
- Python checks them top to bottom and runs the first one that matches
- You can also group types: except (TypeError, ValueError):
- Catch specific exceptions rather than a bare except: — it hides bugs by swallowing everything, including typos

LIVE DEMO

Multiple except Clauses

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Cannot divide by zero")
    except TypeError:
        print("Invalid operand types")

print(safe_divide(10, 2))
print(safe_divide(10, 0))
print(safe_divide(10, "2"))
```

OUTPUT

```
5.0
Cannot divide by zero
None
Invalid operand types
None
```

- Each call is checked against the except clauses independently
- When an except clause runs but has no return, the function implicitly returns None



FULL STRUCTURE

else and finally

- else: runs ONLY if the try block completed with no exception raised
- finally: always runs — whether an exception occurred or not, and even if the try block returns
- finally is the natural place for cleanup: closing files, releasing locks, disconnecting from a database
- Full order: try → except (if error) OR else (if no error) → finally (always)

LIVE DEMO

Complete try / except / else / finally

```
try:
    num = int("42")
except ValueError:
    print("Conversion failed")
else:
    print("Conversion succeeded:", num)
finally:
    print("Execution complete")
```

OUTPUT

```
Conversion succeeded: 42
Execution complete
```

- int("42") succeeds, so except is skipped and else runs
- finally runs regardless of the outcome



SIGNALLING ERRORS

Raising Exceptions Deliberately

- The raise keyword lets YOUR code trigger an exception when it detects an invalid condition
- raise ValueError("message") stops execution and hands control to the nearest matching except
- Useful for validating function arguments before doing real work
- A raised exception propagates upward until some caller catches it — or the program terminates

LIVE DEMO

Using raise for Validation

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print("Caught:", e)
```

OUTPUT

```
Caught: Age cannot be negative
```

- as e captures the exception object so its message can be inspected or logged



CUSTOM ERRORS

Defining Your Own Exception Classes

- Custom exceptions are created by subclassing (usually) Exception
- They give domain-specific errors a clear, self-documenting name
- class InsufficientFundsError(Exception): pass — often just needs the class header
- Callers can catch your custom exception exactly like a built-in one

LIVE DEMO

A Custom Exception Class

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the balance."""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(
            f"Cannot withdraw {amount}, balance is {balance}")
    return balance - amount

try:
    withdraw(500, 800)
except InsufficientFundsError as e:
    print("Transaction failed:", e)
```

OUTPUT

```
Transaction failed: Cannot withdraw
800, balance is 500
```

- InsufficientFundsError reads far more clearly at the call site than a generic ValueError would

PRACTICAL DEMO

Real-World Use — Safe File Handling

```
try:
    with open("marks.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("marks.txt not found - using default data")
    data = "no data"
finally:
    print("File operation attempted")

print("Data:", data)
```

OUTPUT

```
marks.txt not found - using default data
File operation attempted
Data: no data
```

- File I/O is one of the most common real-world sources of exceptions
- finally guarantees the "attempted" message prints whether or not the file existed

RECAP

Section 2 Recap — Exception Handling

- ✓ Syntax errors break the code itself; exceptions occur during execution and can be handled
- ✓ try / except isolates risky code and responds to specific failure types
- ✓ else runs on success; finally always runs, ideal for cleanup
- ✓ raise lets your own code signal an error condition explicitly
- ✓ Custom exception classes make domain errors self-documenting and easy to catch

SECTION 03

Python Regular Expressions

Pattern matching text with the re module





THE RE MODULE

What Are Regular Expressions?

- A regular expression (regex) is a mini pattern-language for describing text to search for
- Python’s built-in re module provides functions to apply these patterns to strings
- Used everywhere: form validation, log parsing, data cleaning, find-and-replace
- Key functions: re.match(), re.search(), re.findall(), re.sub(), re.compile()

Import first

import re

Regex support is part of the standard library — no installation needed

BUILDING BLOCKS

Anatomy of a Pattern

```
^[6-9]\d{9}$
```

`^`	`[6-9]`	`\d`	`{9}`	`$`
Anchor: start of string	Character class: one digit, 6 through 9	Shorthand class: any single digit 0-9	Quantifier: exactly 9 repetitions of \d	Anchor: end of string

Together this pattern validates a 10-digit Indian mobile number starting with 6, 7, 8, or 9.

SYNTAX REFERENCE

Character Classes & Quantifiers

Character Classes

- ✓ \d — any digit (0-9)
- ✓ \w — any word character (letters, digits, _)
- ✓ \s — any whitespace character
- ✓ [abc] — any one of a, b, or c
- ✓ [^abc] — any character EXCEPT a, b, c
- ✓ . — any character except newline

Quantifiers

- ✓ * — zero or more repetitions
- ✓ + — one or more repetitions
- ✓ ? — zero or one (optional)
- ✓ {n} — exactly n repetitions
- ✓ {n,m} — between n and m repetitions
- ✓ ^ and $ — start and end of string anchors

LIVE DEMO

match() vs. search() vs. findall()

```
import re

text = "Student ID: 2021, Marks: 88"

m = re.match(r"Student ID: \d+", text)
s = re.search(r"\d+", text)
f = re.findall(r"\d+", text)

print("match :", m.group())
print("search:", s.group())
print("findall:", f)
```

OUTPUT

```
match : Student ID: 2021
search: 2021
findall: ['2021', '88']
```

- match() checks only from the START of the string
- search() finds the first match ANYWHERE; findall() returns EVERY match as a list



EXTRACTION

Groups — Capturing Parts of a Match

- Parentheses ( ) mark a group inside a pattern, letting you pull out individual pieces
- match.group(0) or .group() returns the whole match; .group(1), .group(2)... return each capture group
- Groups make it possible to extract structured pieces (day, month, year) from one matched string
- Named groups (?P<name>...) let you refer to captures by name instead of position

LIVE DEMO

Extracting Date Parts with Groups

```
import re

date = "26-07-2026"
pattern = r"(\d{2})-(\d{2})-(\d{4})"

match = re.match(pattern, date)
print("Full match:", match.group(0))
print("Day:", match.group(1))
print("Month:", match.group(2))
print("Year:", match.group(3))
```

OUTPUT

```
Full match: 26-07-2026
Day: 26
Month: 07
Year: 2026
```

- Each pair of parentheses becomes one numbered group, in left-to-right order



LIVE DEMO

re.sub() — Search and Replace

```
import re

text = "Contact us at admin@college.edu or support@college.edu"
masked = re.sub(r"[\w.-]+@[\w.-]+", "[EMAIL HIDDEN]", text)

print(masked)
```

OUTPUT

```
Contact us at [EMAIL HIDDEN] or
[EMAIL HIDDEN]
```

- re.sub(pattern, replacement, text) swaps EVERY match in the string, not just the first

PRACTICAL DEMO

Practical Validation — Email Format

```
import re

pattern = r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$"
emails = ["prabu@cse.edu", "invalid@@mail",
"student123@amrita.edu"]

for e in emails:
    valid = bool(re.match(pattern, e))
    print(f"{e:25s} -> {'Valid' if valid else 'Invalid'}")
```

OUTPUT

```
prabu@cse.edu             -> Valid
invalid@@mail             -> Invalid
student123@amrita.edu     -> Valid
```

- ^ and $ ensure the ENTIRE string matches the pattern, not just part of it

PRACTICAL DEMO

Practical Validation — Phone Numbers

```
import re

pattern = r"^[6-9]\d{9}$"
numbers = ["9876543210", "1234567890", "98765432"]

for num in numbers:
    ok = re.match(pattern, num)
    print(num, "->", "Valid" if ok else "Invalid")
```

OUTPUT

```
9876543210 -> Valid
1234567890 -> Invalid
98765432 -> Invalid
```

- 1234567890 fails because it starts with 1, not 6-9
- 98765432 fails because it has only 8 digits, not exactly 10



EFFICIENCY

re.compile() and Flags

- re.compile(pattern) pre-compiles a pattern into a reusable object — faster when applied many times (e.g. in a loop)
- compiled = re.compile(r"\d+"); compiled.findall(text)
- Flags modify matching behaviour, passed as a second argument

re.IGNORECASE — case-insensitive matching

re.MULTILINE — ^ and $ match at each line, not just string start/end

RECAP

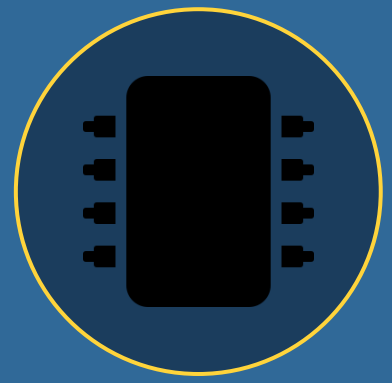
Section 3 Recap — Regular Expressions

- ✓ A regex is a pattern language for describing and matching text, used via the re module
- ✓ match() anchors at the start; search() finds anywhere; findall() returns all matches
- ✓ Parentheses create groups to extract structured pieces of a match
- ✓ re.sub() performs pattern-based search-and-replace across a whole string
- ✓ Anchors (^ $), character classes, and quantifiers combine to build precise validation patterns

SECTION 04

Multithreaded Programming

Running multiple tasks concurrently within a single Python program



Process vs. Thread

Process

own independent memory space



Starting a new PROCESS duplicates memory — heavier, but fully isolated (safer, slower to create).

Process (with multiple threads)

threads SHARE the same memory space



Starting a new THREAD is lightweight and shares memory — faster, but needs careful coordination.



MOTIVATION

Why Use Multithreading?

- Many programs spend time WAITING — for a file to load, a network response, user input
- Multithreading lets a program keep doing useful work while one part waits
- Best suited to I/O-bound tasks: file access, network calls, database queries
- Python’s threading module provides the Thread class to run functions concurrently

Note: for CPU-bound work, the GIL (next slides) limits true parallel speed-up from threads

LIVE DEMO

Creating and Starting Threads

```
import threading
import time

def greet(name):
    print(f"Hello, {name}!")
    time.sleep(1)
    print(f"{name} done")

t1 = threading.Thread(target=greet, args=("Alice",))
t2 = threading.Thread(target=greet, args=("Bob",))

t1.start()
t2.start()

print("Main thread continues...")
```

OUTPUT

```
Hello, Alice!
Hello, Bob!
Main thread continues...
Alice done
Bob done
```

- target= is the function to run; args= is a tuple of arguments passed to it
- The exact interleaving of output can vary between runs — that's concurrency



SYNCHRONIZING

join() — Waiting for a Thread to Finish

- thread.start() launches a thread but does NOT wait for it to complete
- thread.join() blocks the calling code until that thread has fully finished
- Common pattern: start all threads first, then join() all of them, so they truly run concurrently
- Without join(), the main program might exit before background threads finish their work

LIVE DEMO

Waiting on Threads with join()

```
import threading, time

def worker(n):
    time.sleep(n)
    print(f"Worker slept {n}s")

threads = [threading.Thread(target=worker, args=(i,))
           for i in range(1, 4)]

for t in threads:
    t.start()
for t in threads:
    t.join()

print("All workers finished")
```

OUTPUT

```
Worker slept 1s
Worker slept 2s
Worker slept 3s
All workers finished
```

- All three threads start almost immediately and run concurrently
- "All workers finished" only prints after every thread has completed

The Global Interpreter Lock (GIL)

The GIL allows only ONE thread to execute Python bytecode at any instant, even on a multi-core CPU.



Only one green (active) segment exists at any moment across all lanes — the GIL passes between threads in turn.

Practical effect: threading gives concurrency for I/O-bound tasks, but not true parallel speed-up for CPU-bound number crunching (use multiprocessing for that instead).



THE PROBLEM

Race Conditions

- A race condition happens when multiple threads read and modify shared data at the same time
- counter += 1 looks atomic, but is really three steps: read counter, add 1, write it back
- Two threads can interleave these steps and overwrite each other’s update — updates get LOST
- The result becomes unpredictable and depends on timing, making these bugs hard to reproduce and debug

LIVE DEMO

Demonstrating a Race Condition

```
import threading

counter = 0

def increment():
    global counter
    for _ in range(100000):
        counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
t1.start(); t2.start()
t1.join();  t2.join()

print("Expected: 200000, Got:", counter)
```

OUTPUT

```
Expected: 200000, Got: 187342
```

- Two threads increment the SAME counter 100,000 times each with no coordination
- The exact "Got" value varies each run — and is almost always less than 200000



THE FIX

Locks — Protecting Shared Data

- threading.Lock() creates a lock object that only one thread can "hold" at a time
- with lock: … ensures the enclosed block runs as one uninterruptible unit (a critical section)
- Other threads trying to acquire the same lock simply wait their turn
- Trade-off: locks fix correctness but reduce concurrency — only lock the smallest section necessary

LIVE DEMO

Fixing the Race Condition with Lock

```
import threading

counter = 0
lock = threading.Lock()

def safe_increment():
    global counter
    for _ in range(100000):
        with lock:
            counter += 1

t1 = threading.Thread(target=safe_increment)
t2 = threading.Thread(target=safe_increment)
t1.start(); t2.start()
t1.join();  t2.join()
print("Expected: 200000, Got:", counter)
```

OUTPUT

```
Expected: 200000, Got: 200000
```

- with lock: guarantees only one thread executes counter += 1 at a time
- The result is now correct and repeatable on every run



HOUSEKEEPING

Daemon Threads & Thread Pools

- A daemon thread (thread.daemon = True) is killed automatically when the main program exits
- Useful for background tasks that should never block the program from closing (e.g. logging)
- Manually creating and joining many threads gets repetitive — ThreadPoolExecutor manages a reusable pool
- from concurrent.futures import ThreadPoolExecutor — a higher-level, modern alternative to raw Thread objects

LIVE DEMO

ThreadPoolExecutor — a Higher-Level API

```
from concurrent.futures import ThreadPoolExecutor

def square(n):
    return n * n

with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(square, [1, 2, 3, 4, 5])

print(list(results))
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

- executor.map() distributes the five calls across (up to) 3 worker threads automatically
- The with block ensures all threads are cleanly shut down afterward

RECAP

Section 4 Recap — Multithreading

- ✓ Threads share memory within one process; processes are isolated and heavier to create
- ✓ The GIL allows only one thread to run Python bytecode at a time — threads suit I/O-bound work
- ✓ thread.start() launches a thread; thread.join() waits for it to finish
- ✓ Race conditions arise when threads update shared data without coordination
- ✓ threading.Lock() protects critical sections; ThreadPoolExecutor simplifies managing many threads

MODULE SUMMARY

From Objects to Concurrency



OOP

Model real-world entities as classes and objects; reuse via inheritance



Exceptions

Handle failure gracefully with try / except / else / finally and custom errors



Regex

Match, extract, and validate text patterns using the re module



Threading

Run concurrent tasks safely with threads, locks, and thread pools

Reference: Allen B. Downey, "Think Python: How to Think Like a Computer Scientist," 2nd Edition